

Time/space complexity

COMPLETE LEARNING GUIDE

What You'll Learn

Big-O notation is the common language interviewers use to evaluate whether a solution is actually good, independent of the specific programming language or hardware it runs on — and being unable to state your own solution's complexity correctly, unprompted, is one of the fastest ways to lose credibility in a technical interview even when the code itself works. This guide covers Big-O, Big-Theta, and Big-Omega formally but briefly, the common complexity classes ranked from best to worst with concrete examples of each, how to analyze nested loops and recursive functions (including recurrence relations) to derive their complexity, the distinction between auxiliary space and total space (and why recursive call-stack depth counts as real space), amortized analysis using ArrayList's dynamic resizing as the running example, and a practical reference table mapping common Java operations to their actual complexity.

1. Big-O, Big-Theta, and Big-Omega

Big-O (O) describes an upper bound on growth rate — "this algorithm never does worse than this" — and is what's almost universally meant in casual interview conversation when someone says "the complexity is $O(n)$." Big-Omega (Ω) describes a lower bound — the best case, or a guarantee the algorithm can't do better than. Big-Theta (Θ) describes a tight bound where the upper and lower bounds match, meaning the algorithm's growth rate is precisely characterized, not just bounded from one side. In practice, most interview conversations use "Big-O" loosely to mean the tight bound (Θ), Ω for the best case, and that casual usage is fine as long as you understand what the more precise notation actually distinguishes if asked directly.

2. Common Complexity Classes, Ranked

Recognizing which complexity class a piece of code falls into at a glance — from the structure of its loops and recursive calls, without needing to trace through execution — is a core interview skill, and the classes below are worth memorizing in order alongside a canonical example of each.

- $O(1)$ constant — array index access, HashMap get/put (average case).
- $O(\log n)$ logarithmic — binary search, balanced binary search tree operations.
- $O(n)$ linear — a single pass through an array, linear search.

- $O(n \log n)$ linearithmic — efficient comparison-based sorting (mergesort, heapsort, Java's `Arrays.sort` for objects).
- $O(n^2)$ quadratic — nested loops over the same input, naive pair-checking, bubble/insertion/selection sort.
- $O(2^n)$ exponential — naive recursive Fibonacci, generating all subsets of a set.
- $O(n!)$ factorial — generating all permutations of a set, brute-force traveling salesman.

3. Analyzing Loops and Nested Loops

A single loop that runs n times, doing $O(1)$ work per iteration, is $O(n)$ overall — the complexity is the iteration count times the per-iteration cost. Two independent (sequential, not nested) loops each running n times is still $O(n) + O(n) = O(2n)$, which simplifies to $O(n)$, since Big-O drops constant multipliers. Two nested loops, where the inner loop runs n times for each of the outer loop's n iterations, is $O(n) \times O(n) = O(n^2)$ — the multiplication (not addition) is the key insight, and it generalizes directly: k levels of nesting, each running n times, gives $O(n^k)$.

JAVA

```
// O(n) -- single pass
for (int i = 0; i < n; i++) { /* O(1) work */ }

// O(n) -- two SEQUENTIAL loops, not nested: O(n) + O(n) = O(2n) = O(n)
for (int i = 0; i < n; i++) { /* ... */ }
for (int j = 0; j < n; j++) { /* ... */ }

// O(n^2) -- NESTED: inner loop runs n times for EACH of the outer loop's n
// iterations
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) { /* O(1) work */ }
}
```

4. Analyzing Recursive Complexity with Recurrence Relations

A recursive function's time complexity is expressed as a recurrence relation — an equation defining the cost of solving a problem of size n in terms of the cost of solving smaller subproblems, plus the work done to combine them. `factorial(n)` calls itself once on a problem of size $n-1$ with $O(1)$ extra work, giving $T(n) = T(n-1) + O(1)$, which resolves to $O(n)$. Binary search calls itself once on a problem of HALF the size with $O(1)$ extra work, giving $T(n) = T(n/2) + O(1)$, which resolves to $O(\log n)$ — the halving is exactly why binary search is logarithmic. A classic divide-and-conquer sort like mergesort calls itself TWICE on half-sized problems, plus $O(n)$ work to merge the results, giving $T(n) = 2T(n/2) + O(n)$, which resolves to $O(n \log n)$ via the Master Theorem.

5. Auxiliary Space vs. Total Space, and Recursive Stack Space

Space complexity has two components worth distinguishing precisely: input space (memory the input itself already occupies, usually not counted) and auxiliary space (extra memory the algorithm allocates beyond the input — new arrays, hash maps, or recursive call-stack frames). A recursive function's auxiliary space includes its maximum call-stack depth, since every active call's frame occupies real memory simultaneously — naive recursive `factorial(n)` is $O(n)$ auxiliary space from n stack frames, even though it allocates no explicit data

structure at all, a point candidates frequently miss when asked for space complexity.

6. Amortized Analysis: ArrayList's Dynamic Resizing

Amortized analysis answers "what's the average cost per operation across a long sequence of operations," even when individual operations vary wildly in cost — the canonical example is `ArrayList.add()`, which is usually $O(1)$ (there's room in the backing array) but occasionally $O(n)$ (the backing array is full and must be reallocated to a larger size, with every existing element copied over). Because array doubling means the expensive $O(n)$ resize happens exponentially less often as the list grows, the total cost of n additions sums to $O(n)$, making the amortized cost per `add()` exactly $O(1)$ — this is why "ArrayList.add() is $O(1)$ " is the standard, correct answer despite individual worst-case calls being $O(n)$.

7. A Practical Complexity Reference Table

Interviewers expect instant recall of these specific numbers for Java's standard library types, since misstating one (like claiming `ArrayList.contains()` is $O(1)$) is a red flag that undermines confidence in every other claim you make about a solution's efficiency.

- `ArrayList`: `get(i)` $O(1)$, `add(end)` $O(1)$ amortized, `add(index)/remove(index)` $O(n)$, `contains()` $O(n)$.
- `LinkedList`: `get(i)` $O(n)$, `add/remove` at a known node $O(1)$, `contains()` $O(n)$.
- `HashMap/HashSet`: `get/put/contains` $O(1)$ average, $O(n)$ worst case (before Java 8 treeification), $O(\log n)$ worst case since Java 8.
- `TreeMap/TreeSet`: `get/put/contains` $O(\log n)$, always — no average/worst-case distinction due to the underlying balanced tree.
- `Arrays.sort()`: $O(n \log n)$ for objects (a stable, TimSort-derived algorithm); Dual-Pivot Quicksort $O(n \log n)$ average for primitives.
- `Arrays.binarySearch()`: $O(\log n)$, but only valid on an already-sorted array.

Common Pitfalls

- Stating a nested loop's complexity as $O(n)$ instead of $O(n^2)$ by forgetting that nested (not sequential) loops multiply, not add.
- Forgetting that a recursive function's call-stack depth counts as auxiliary space, understating an algorithm's actual space complexity.
- Claiming `ArrayList.add()` is "always $O(1)$ " without the word "amortized," missing the occasional $O(n)$ resize cost that the amortized analysis accounts for.
- Claiming `HashMap` operations are "always $O(1)$ " without qualifying "average case" — worst case is $O(\log n)$ since Java 8's treeification (or $O(n)$ before it).
- Confusing $O(n \log n)$ and $O(n^2)$ when quickly assessing a nested-loop-plus-sort solution, especially under interview time pressure.

- Assuming Big-O captures real-world performance directly — an $O(n)$ algorithm with a large constant factor can be slower in practice than an $O(n \log n)$ algorithm with a small one, for realistic input sizes.
- Not simplifying complexity expressions — writing $O(2n + 3)$ instead of the simplified $O(n)$, which is what interviewers expect as the final answer.

Key Takeaways

- Big-O is an upper bound, Big-Omega a lower bound, Big-Theta a tight bound — casual interview usage of "Big-O" usually means the tight bound of the typical/worst case.
- Sequential loops add their complexities ($O(n) + O(n) = O(n)$); nested loops multiply them ($O(n) \times O(n) = O(n^2)$).
- A recursive function's complexity is found by writing its recurrence relation — $T(n) = T(n/2) + O(1)$ gives $O(\log n)$; $T(n) = 2T(n/2) + O(n)$ gives $O(n \log n)$.
- Auxiliary space includes recursive call-stack depth, not just explicitly allocated data structures.
- Amortized analysis explains why `ArrayList.add()` is correctly described as $O(1)$ despite occasional $O(n)$ resize operations.
- Memorize the standard Java collection complexity table (`ArrayList`, `LinkedList`, `HashMap`, `TreeMap`) cold — it's assumed baseline knowledge in interviews.

Learn More

GeeksforGeeks — Analysis of Algorithms (Big-O, Big-Omega, Big-Theta)

<https://www.geeksforgeeks.org/dsa/analysis-of-algorithms/>

GeeksforGeeks — Time Complexity of Java Collections

<https://www.geeksforgeeks.org/java/time-complexities-of-different-data-structures-in-java/>

GeeksforGeeks — Amortized Analysis

<https://www.geeksforgeeks.org/dsa/amortized-analysis-introduction/>